Software Change Management Standard - (SCMS)

Software Change Management Standard - (SCMS)

OOF® Origin Open Foundation®

Independent Methodological Authority

Architecture: SLGA™ — Software Lifecycle Governance Architecture

Category: Governance & Enforcement

Subcategory: Software Lifecycle Governance

Type: Parent Standard

Governed Space: Software Change Management Governance

Standard ID: SLGA-SCMS-09

Origin ID: OOF-SLGA-SCMS

Version: 1.0

Status: Canonical Open

Effective Date: 14.10.2026

Compatibility: Methodology OS — Modular Compatibility

GIE-Compatible: Yes

AI-Readable: Yes

Authority: OOF® — OriginOpen® Foundation

Protection: Protected Methodological Work

Canonical Language: English

Canonical Definition

Software Change Management Standard (SCMS) is the Parent Standard within SLGA™ that governs how proposed, required, and emerging changes to software are identified, evaluated, coordinated, authorized through applicable governance mechanisms, directed into the appropriate software lifecycle processes, verified against their intended outcomes, and formally closed or retained as unresolved obligations.

SCMS establishes software change as a governed transition between

identifiable software conditions rather than merely an edit to source code, a deployment request, or an operational modification.

It requires changes to remain associated with their origin, purpose,

affected software objects, current state, intended state, impact, dependencies, applicable authority, lifecycle requirements, implementation evidence, and resulting operational consequences.

The standard governs changes arising from defects, security

vulnerabilities, functional requirements, dependency updates, configuration modifications, operational incidents, infrastructure transitions, regulatory requirements, automated recommendations, and autonomous software agents.

SCMS recognizes that different changes require different lifecycle pathways and that a change cannot legitimately bypass necessary requirements, creation, dependency, configuration, quality, security, release, or deployment governance merely because it is urgent, automated, or technically executable.

Its principal governance output is the Software Change Governance Profile.

Operational Role

SCMS establishes the governance structure connecting a recognized need for

software change with the lifecycle processes required to evaluate, implement, verify, and close that change.

It governs:

• Software change recognition

• Change request identity

• Change origin and justification

• Change purpose and intended outcome

• Affected software objects

• Current and proposed software states

• Change classification

• Change impact assessment

• Change feasibility and dependencies

• Change scope and boundaries

• Change coordination

• Applicable authority and approval references

• Lifecycle re-entry

• Emergency change conditions

• Automated and autonomous changes

• Implementation traceability

• Change outcome verification

• Residual obligations

• Change closure and continuity

SCMS does not replace the specialized governance mechanisms responsible for software creation, quality verification, security, release authorization, or deployment execution.

It governs how a change moves through those mechanisms and whether the resulting change remains identifiable, justified, coordinated, and verifiably resolved.

Core Question

How can software changes remain identifiable, justified, coordinated, authorized, traceable, and verifiably resolved throughout the software lifecycle when change requirements, affected components, dependencies, operational conditions, implementation actors, and resulting software states may continuously evolve?

Position in Architecture Governance Flow

SLGA™ Governance Flow

Software Requirements → Software Creation → Software Dependencies →

Software Configuration → Software Quality → Software Security → Software Release → Software Deployment → Software Change Management → Software Retirement

Cross-cutting: Software Lifecycle Control

SCMS occupies the ninth position in the SLGA™ architecture.

Its position does not mean that software changes occur only after deployment.

Software change may arise during any lifecycle stage.

SCMS governs the recognition, assessment, coordination, and resolution of

those changes and directs affected work back into the appropriate lifecycle processes.

Change Recognition → Change Assessment → Lifecycle Re-entry → Governed

Implementation → Outcome Verification → Change Closure

SCMS therefore provides a governed change-management mechanism across the

software lifecycle without replacing the individual Parent Standards.

• Governed Space

Software Change Management Governance

This space governs the formation, assessment, coordination, lifecycle

routing, implementation traceability, verification, and closure of software changes.

It begins when a potential or required change becomes identifiable.

It continues while the change is under assessment, approved for applicable lifecycle processing, implemented, verified, deferred, rejected, superseded, or maintained as an unresolved obligation.

The governed space includes changes initiated by humans, organizations,

automated systems, AI agents, external dependencies, security events, and operational conditions.

It ends for an individual change only when an appropriate closure or disposition state has been established and supported by evidence.

Governed Object

The primary Governed Object is:

A Software Change Case representing the identifiable and

evidence-supported relationship between a proposed or required software change, its originating condition, affected software objects, intended outcome, assessed impact, applicable governance requirements, lifecycle pathway, implementation references, resulting state, and closure decision.

The governed object must preserve, where applicable:

Change Case ID

Change Request ID

Software Product ID

Change Origin

Change Purpose

Affected Software Objects

Current State Reference

Intended State Reference

Change Classification

Impact Assessment Reference

Applicable Authority Reference

Lifecycle Re-entry Reference

Implementation References

Verification Evidence

Residual Conditions

Change Status

Version

Scope

SCMS governs software changes involving:

Functional Behavior

Software Requirements

Source Code

Software Architecture

Dependencies

Build and Configuration

Quality Conditions

Security Conditions

Software Releases

Deployment Conditions

Operational Software States

Interfaces and Integrations

Data Compatibility

Automation Logic

Autonomous Agent Behavior

Software Retirement Dependencies

Its applicability depends on the nature and consequences of the change

rather than the particular technology used.

Foundational Principle

A software change must remain governed by what it alters, why the alteration is necessary, which lifecycle mechanisms are affected, and whether the resulting state can be legitimately and verifiably established.

A small code modification may produce significant operational consequences.

A dependency update may require security, quality, configuration, and release reassessment.

An emergency correction may still require deployment authorization.

An autonomous agent may propose a useful modification without possessing authority to introduce it into production.

Change Implemented ≠ Change Governed.

Change Approved ≠ Every Lifecycle Requirement Satisfied.

Technical Modification ≠ Verified Software Improvement.

Software Change Identity

Every governed software change must have a stable identity.

Relevant elements include:

Change Case ID

Change Request ID

Software Product ID

Change Origin

Affected Object References

Change Owner or Responsible Function

Creation Time

Current Status

Change Version

A change may evolve through multiple assessments and implementation attempts without losing its historical identity.

Change Origin

Software changes may originate from:

New Requirements

Defect Reports

Security Findings

Operational Incidents

Dependency Changes

Configuration Changes

Performance Findings

Compatibility Requirements

Infrastructure Changes

Regulatory or Contractual Requirements

Automated Analysis

Autonomous Agent Recommendations

Retirement or Migration Requirements

The origin must remain identifiable and distinguishable from the actor proposing or implementing the change.

Change Purpose

Every change must have an identifiable purpose.

Examples include:

Correct a Defect

Introduce Functionality

Remove Functionality

Improve Performance

Address a Vulnerability

Restore Compatibility

Update a Dependency

Modify Configuration

Reduce Operational Risk

Support Migration

Restore Operational Capability

Prepare Software Retirement

A proposed technical modification without a sufficiently defined purpose

may be unsuitable for governance assessment.

Change Justification

Change justification explains why the proposed modification should be considered.

Relevant elements include:

Identified Problem or Opportunity

Supporting Evidence

Expected Benefit

Operational Necessity

Applicable Requirement

Urgency

Alternatives Considered

Consequences of No Change

Known Uncertainty

Justification must not be confused with authorization.

A well-supported need for change does not automatically authorize implementation.

Current Software State

SCMS requires a suitable reference to the software state before the

proposed change.

Relevant elements include:

Current Requirements

Current Software Artifacts

Current Dependencies

Current Configuration

Current Quality Conditions

Current Security Conditions

Current Release State

Current Deployment State

Known Defects or Restrictions

The necessary baseline depends on the scope and impact of the proposed

change.

Intended Software State

The intended state identifies what the change is expected to establish.

Relevant elements include:

Target Functionality

Target Software Behavior

Target Artifact or Configuration

Target Dependency Conditions

Target Quality Requirements

Target Security Conditions

Target Compatibility

Target Operational Outcome

Verification Criteria

The intended state must remain distinguishable from the state actually produced by implementation.

Change Scope

Change scope identifies the objects and conditions intended to be modified.

Relevant dimensions include:

Affected Components

Affected Interfaces

Affected Dependencies

Affected Configurations

Affected Environments

Affected Data Structures

Affected Operational Functions

Affected Users or Systems

Excluded Elements

Scope must be sufficiently defined to support impact assessment and lifecycle coordination.

Change Boundary

The change boundary identifies the permitted extent of the proposed

modification and the conditions requiring additional assessment.

Relevant boundaries include:

Software Object Boundary

Functional Boundary

Dependency Boundary

Configuration Boundary

Security Boundary

Environment Boundary

Data Boundary

Authority Boundary

Lifecycle Boundary

Operational Impact Boundary

A change request must not become an unrestricted permission to modify related software or infrastructure.

Change Classification

SCMS must distinguish changes according to their governance-relevant characteristics.

Potential classifications include:

Functional Change

Corrective Change

Security Change

Dependency Change

Configuration Change

Compatibility Change

Performance Change

Infrastructure-Related Software Change

Emergency Change

Migration Change

Retirement-Related Change

Automated Change

Autonomous Agent-Initiated Change

Multiple classifications may apply to one change.

Classification must not replace impact assessment.

Change Significance

Change significance concerns the potential consequences of the modification.

Relevant dimensions include:

Functional Impact

Operational Impact

Security Impact

Data Impact

Dependency Impact

Infrastructure Impact

User Impact

Reversibility

Verification Complexity

Recovery Consequences

A change with few modified code lines may still have high governance significance.

Change Impact Assessment

SCMS requires assessment of material consequences before a change proceeds

through applicable lifecycle mechanisms.

Impact assessment may include:

Affected Software Objects

Affected Requirements

Affected Dependencies

Affected Configurations

Affected Quality Conditions

Affected Security Conditions

Affected Releases

Affected Deployments

Affected Operational Environments

Affected Recovery Conditions

The assessment must preserve known limitations and unresolved impact questions.

Direct and Indirect Impact

Change assessment must distinguish direct modifications from indirect consequences.

Direct Impact

The software objects or conditions intentionally modified.

Indirect Impact

Other objects, functions, dependencies, environments, or operational

conditions affected through the modification.

A change affecting one component may create consequences across multiple services or lifecycle stages.

Limited Change Scope ≠ Limited Change Impact.

Dependency Impact

Software changes may alter dependency relationships.

Relevant considerations include:

Introduced Dependencies

Removed Dependencies

Dependency Version Changes

Transitive Dependencies

Compatibility Requirements

Dependency Availability

Security Exposure

Continuity Requirements

SCMS identifies the change-related impact.

SDS governs the underlying dependency mechanisms.

Configuration Impact

Changes may affect software configuration and build conditions.

Relevant elements include:

Configuration Baseline

Build Inputs

Artifact Identity

Environment Parameters

Runtime Configuration

Configuration Compatibility

Reproducibility Requirements

Configuration Restoration

SCMS coordinates applicable configuration governance through SCFS.

Quality Impact

Change assessment must identify relevant quality consequences.

These may include:

Functional Correctness

Reliability

Performance

Compatibility

Maintainability

Test Coverage

Regression Exposure

Known Defects

Quality Verification Requirements

SQS governs the applicable quality assessment and verification mechanisms.

Security Impact

Software changes may introduce, remove, or alter security conditions.

Relevant elements include:

Attack Surface

Access Permissions

Authentication Behavior

Authorization Logic

Data Exposure

Dependency Vulnerabilities

Security Configuration

Secret Handling

Security Testing Requirements

Remediation Obligations

SSS governs the applicable software security mechanisms.

Operational Impact

SCMS must assess the potential operational consequences of a change.

Relevant dimensions include:

Service Availability

Deployment Complexity

Resource Consumption

Data Integrity

Infrastructure Compatibility

Monitoring Requirements

Recovery Feasibility

User Disruption

Physical-System Behavior Where Applicable

Operational impact must not be inferred solely from the technical size of the modification.

Change Feasibility

Change feasibility concerns whether the proposed modification can reasonably be implemented and verified under applicable conditions.

Relevant factors include:

Technical Capability

Required Resources

Required Expertise

Dependency Availability

Infrastructure Constraints

Verification Capability

Time Constraints

Operational Restrictions

Recovery Capability

Authority Requirements

Feasibility does not independently establish legitimacy or desirability.

Change Alternatives

Where material consequences justify comparison, SCMS should preserve relevant alternatives.

These may include:

Implement Proposed Change

Implement a Different Solution

Restrict Existing Functionality

Apply a Temporary Mitigation

Defer Implementation

Reject the Change

Retire or Replace the Affected Software

Alternative assessment must consider the actual problem and applicable governance requirements.

Change Priority and Urgency

SCMS must distinguish change priority from change authorization.

Relevant factors include:

Operational Severity

Security Exposure

Business Criticality

User Impact

Dependency Deadline

Regulatory Requirement

Recovery Necessity

Implementation Risk

Urgency may affect coordination and permitted procedures but does not independently create authority.

Urgent Change ≠ Automatically Authorized Change.

Change Coordination

Software changes may require coordination among multiple functions.

Relevant elements include:

Affected Teams

Responsible Functions

Required Reviews

Dependency Coordination

Configuration Coordination

Quality Verification

Security Review

Release Planning

Deployment Planning

Operational Communication

Coordination requirements must reflect the change's scope and consequences.

Change Authority

SCMS must identify the applicable authority references governing change decisions.

Relevant elements include:

Change Decision Authority

Approval Scope

Decision Actor

Delegation Conditions

Applicable Restrictions

Validity

Required Escalation

Decision Evidence

Change approval must remain distinct from release authorization and deployment authorization.

Change Decision

SCMS recognizes the following principal decisions:

Accepted for Lifecycle Processing

Accepted With Conditions

Deferred

Rejected

Reassessment Required

Withdrawn

Acceptance for lifecycle processing does not mean that the proposed change has already satisfied all implementation, verification, release, or deployment requirements.

Conditional Change Acceptance

A change may be accepted subject to explicit conditions.

Examples include:

Additional Impact Assessment

Security Verification

Dependency Compatibility Evidence

Required Test Coverage

Independent Review

Restricted Implementation Scope

Additional Deployment Conditions

Recovery Planning

Conditions must remain traceable to the relevant lifecycle mechanism.

Lifecycle Re-entry

Software change may require returning to one or more earlier lifecycle governance stages.

Examples include:

Requirements Reassessment

Software Creation

Dependency Assessment

Configuration Revision

Quality Verification

Security Verification

Release Formation

Deployment Planning

Lifecycle re-entry must be determined by the nature and consequences of the change.

It must not be reduced to a fixed sequence applied identically to every modification.

Lifecycle Re-entry Principle

A software change must re-enter every lifecycle governance mechanism materially affected by the change, but it should not be forced through unrelated mechanisms without a justified governance need.

This principle preserves proportionality without weakening necessary

control.

A documentation correction may require limited lifecycle processing.

A dependency update may require configuration, quality, security, release, and deployment reassessment.

A modification to autonomous system behavior may require additional authority, safety, and operational verification.

Change Size ≠ Required Governance Depth.

Lifecycle Re-entry Determination

Re-entry determination must identify:

Affected Lifecycle Stages

Reason for Re-entry

Required Governance Activities

Applicable Standards

Existing Evidence Reusability

Evidence Requiring Renewal

Required Decisions

Completion Conditions

A previously completed lifecycle assessment may remain applicable where its assumptions and evidence remain valid.

Existing Evidence Reuse

SCMS permits reuse of existing evidence when it remains applicable.

Relevant considerations include:

Evidence Scope

Affected Software Object

Change Impact

Evidence Freshness

Unchanged Assumptions

Dependency Conditions

Verification Coverage

Known Limitations

Evidence reuse must be justified rather than assumed.

Previously Verified ≠ Still Verified After Material Change.

Change Implementation Reference

SCMS must preserve traceability to the mechanisms performing the change.

Relevant elements include:

Implementation Work Item

Affected Artifact

Implementation Actor

Development Reference

Configuration Reference

Dependency Reference

Verification Reference

Release Reference

Deployment Reference

SCMS coordinates implementation evidence without replacing the governance

of individual technical activities.

Change Implementation Boundaries

Implementation must remain within the assessed and legitimately permitted scope.

Material deviations may include:

Additional Component Changes

Unplanned Dependency Introduction

New Data Modification

Unexpected Infrastructure Action

Unapproved Configuration Change

Expanded Operational Scope

Changed Execution Actor

New Security Exposure

Material deviations require reassessment or additional authorization where applicable.

Automated Change Management

Automated systems may generate, classify, route, or implement software changes.

SCMS must preserve:

Automation Identity

Assigned Function

Change Origin

Decision Policy Reference

Permitted Actions

Affected Objects

Applicable Constraints

Verification Requirements

Escalation Conditions

Execution Evidence

Automation must not convert technical modification capability into unrestricted change authority.

Autonomous Agent-Initiated Change

Autonomous AI agents may identify defects, propose code changes, modify dependencies, or recommend operational corrections.

SCMS must distinguish:

Agent Recommendation

Change Request

Impact Assessment

Change Decision

Authorized Implementation

Verified Outcome

An agent's confidence in a proposed solution does not establish that the solution is correct, safe, authorized, or suitable for production.

Autonomous Execution Boundary

Where autonomous agents implement changes, SCMS must preserve:

Agent Identity

Assigned Objective

Permitted Software Objects

Permitted Tools

Data Access Boundaries

Infrastructure Access Boundaries

Required Reviews

Escalation Conditions

Human Override Requirements

Evidence

Task Authority ≠ Constituent Action Authority.

A software change objective does not establish permission for every tool call, repository modification, dependency installation, or deployment action used to achieve it.

Emergency Change Governance

Emergency changes may be necessary to address critical vulnerabilities, operational failures, or other urgent conditions.

SCMS must preserve:

Emergency Trigger

Urgency Justification

Applicable Emergency Authority

Permitted Scope

Mandatory Safeguards

Temporary Exceptions

Required Verification

Deployment Conditions

Post-Implementation Review

Emergency processing may accelerate applicable governance procedures without eliminating the need for legitimate authority and operational evidence.

Change Exception Governance

SCMS must identify exceptions to applicable change-management requirements.

Relevant elements include:

Exception ID

Affected Requirement

Reason

Competent Authority

Permitted Deviation

Conditions

Validity

Residual Obligations

Review Requirement

An exception must not be treated as legitimate merely because it appears in a change record.

Change Conflict

A change conflict exists when relevant requirements, decisions, implementation conditions, or evidence are materially inconsistent.

Examples include:

Conflicting Requirements

Incompatible Dependency Changes

Contradictory Security Conditions

Overlapping Changes

Incompatible Configuration Revisions

Competing Release Objectives

Conflicting Deployment Restrictions

Inconsistent Verification Results

Material conflicts must be identified and resolved or explicitly governed before the affected change can proceed.

Concurrent Changes

Multiple software changes may affect the same objects or operational conditions.

SCMS must identify:

Related Change Cases

Shared Components

Overlapping Dependencies

Conflicting Configuration Changes

Execution Order Dependencies

Combined Operational Impact

Coordination Requirements

Verification Implications

Individually acceptable changes may produce unacceptable consequences when combined.

Change Sequence and Dependency

Some changes require a defined implementation order.

Relevant elements include:

Prerequisite Changes

Dependent Changes

Compatibility Transitions

Migration Requirements

Release Dependencies

Deployment Dependencies

Recovery Dependencies

A change must not be treated as independently executable when its success depends on unresolved preceding changes.

Change Traceability

SCMS must preserve the relationship:

Change Origin → Change Purpose → Affected Objects → Impact Assessment →

Change Decision → Lifecycle Re-entry → Implementation → Verification → Resulting State → Closure

This relationship must remain reconstructable across relevant systems, teams, and automated execution paths.

Change Verification

Change verification evaluates whether the intended modification was

implemented and whether the required resulting conditions are supported by evidence.

Relevant elements include:

Change Objective

Implemented Modification

Affected Artifacts

Verification Criteria

Quality Evidence

Security Evidence

Configuration Evidence

Release Evidence

Deployment Evidence

Observed Result

Verification must reflect the actual scope and consequences of the change.

Intended–Actual Change Alignment

SCMS must distinguish the intended change from the modification actually implemented.

Relevant relationships include:

Requested Scope ↔ Implemented Scope

Intended Artifact ↔ Resulting Artifact

Expected Behavior ↔ Verified Behavior

Expected Dependencies ↔ Actual Dependencies

Expected Configuration ↔ Actual Configuration

Expected Operational Outcome ↔ Observed Outcome

Material misalignment must remain visible.

Change Outcome Assessment

A change may produce different outcomes.

Examples include:

Objective Achieved

Objective Partially Achieved

Objective Not Achieved

Unintended Consequences Identified

Verification Incomplete

Outcome Unknown

Additional Change Required

Outcome assessment must not be reduced to whether implementation tasks were marked complete.

Change Failure

Change failure may occur when:

Implementation Cannot Be Completed

Required Verification Fails

New Defects Are Introduced

Security Conditions Become Unacceptable

Dependencies Become Incompatible

Operational Conditions Deteriorate

Intended Outcome Is Not Achieved

Material Scope Violations Occur

A failed change must remain traceable and may require a new change case, reassessment, rollback, or recovery.

Change Reassessment

Reassessment may be required when:

Change Scope Expands

New Impact Evidence Emerges

Dependencies Change

Security Conditions Change

Implementation Deviates

Verification Fails

Operational Conditions Change

Authority Changes

Related Changes Introduce Conflicts

Recovery Becomes Necessary

Reassessment must identify which earlier conclusions remain valid.

Change Version Continuity

SCMS must preserve material revisions to a change case.

Relevant elements include:

Change Case Version

Previous Version

Changed Scope

Changed Purpose

Changed Impact

Changed Decision

Changed Lifecycle Pathway

Changed Verification Requirements

Revision Reason

Revision Evidence

A revised change case must not silently replace the version associated with an earlier decision.

Residual Change Obligations

A software change may leave unresolved obligations.

Examples include:

Deferred Verification

Temporary Configuration

Known Compatibility Limitation

Unresolved Security Condition

Additional Deployment Requirement

Outstanding Documentation

Required Monitoring

Further Migration Work

Future Retirement Dependency

Residual obligations must remain identifiable and assigned to an applicable governance mechanism.

Change Closure

Change closure requires an explicit disposition supported by evidence.

Relevant elements include:

Change Objective

Implementation Status

Verification Results

Resulting Software State

Release or Deployment References

Residual Obligations

Closure Decision

Closure Authority

Closure Time

Closure Evidence

Closure does not require that every proposed change be implemented.

A rejected, withdrawn, or superseded change may be legitimately closed with an appropriate disposition record.

Change Closure Conditions

A change may be closed when its disposition is sufficiently established.

Possible closure conditions include:

Implemented and Verified

Implemented With Accepted Residual Conditions

Rejected With Justification

Withdrawn

Superseded

Transferred to Another Governed Change Case

Resolved Through an Alternative Mechanism

A change must not be classified as successfully implemented merely because the administrative case is closed.

Change Governance Classification

SCMS recognizes the following classifications:

Change Identified

Request Pending

Under Assessment

Impact Assessment Incomplete

Evidence Insufficient

Conflict Identified

Accepted for Lifecycle Processing

Accepted With Conditions

Deferred

Rejected

Implementation Pending

Implementation In Progress

Verification Pending

Partially Verified

Implemented and Verified

Implementation Failed

Reassessment Required

Residual Obligations Outstanding

Withdrawn

Superseded

Closed

Unknown

Conflicted

Classification must remain associated with the relevant change case and supporting evidence.

Core Modules

SCMS is implemented through five Core Modules. 58.1 Change Request Management Module (CRMM)

Governed Space: Software Change Request Governance

CRMM governs the identification, formation, classification, maintenance,

and disposition of software change requests.

It establishes the identifiable relationship between a change origin,

purpose, affected software objects, requested modification, supporting evidence, and initial request status.

Principal Output: Software Change Request Profile

58.2 Change Impact Assessment Module (CIAM)

Governed Space: Software Change Impact Governance

CIAM governs the assessment of direct and indirect consequences associated

with proposed software changes.

It evaluates affected requirements, artifacts, dependencies,

configurations, quality, security, operational conditions, feasibility, uncertainty, and required lifecycle reassessment.

Principal Output: Software Change Impact Profile

58.3 Change Coordination Control Module (CHCM)

Governed Space: Software Change Coordination Governance

CHCM governs the coordination of software changes across affected actors,

software objects, dependencies, implementation activities, decision references, and lifecycle functions.

It preserves change relationships, sequencing, applicable coordination

conditions, and material conflicts.

Principal Output: Software Change Coordination Profile

58.4 Lifecycle Re-entry Control Module (LRECM)

Governed Space: Software Change Lifecycle Re-entry Governance

LRECM governs how a software change is directed into the appropriate SLGA™ lifecycle mechanisms according to its assessed impact and applicable requirements.

It identifies which lifecycle processes must be repeated, renewed, extended, or otherwise revisited and preserves evidence of their completion.

Principal Output: Software Lifecycle Re-entry Profile

58.5 Change Closure Management Module (CCLM)

Governed Space: Software Change Closure Governance

CCLM governs the verification, classification, disposition, and formal

closure of software change cases.

It establishes whether the intended change outcome was achieved, which

evidence supports the conclusion, and which residual obligations or unresolved conditions remain.

Principal Output: Software Change Closure Profile

Parent Standard Governance Flow

Change Origin → Change Recognition → Change Request → Change Purpose →

Affected Software Objects → Change Scope → Impact Assessment → Change Classification → Coordination → Applicable Authority → Change Decision → Lifecycle Re-entry → Governed Implementation → Verification → Outcome Assessment → Residual Obligations → Change Closure

Where material impact is unknown:

Change Request → Evidence Gap → Additional Assessment → Reclassification /

Deferral

Where a change requires additional lifecycle governance:

Change Impact → Affected Lifecycle Standards → Required Re-entry →

Applicable Governance Outputs → Implementation Continuation

Where implementation deviates materially:

Approved Change Scope → Actual Modification → Material Deviation →

Reassessment → Revised Decision / Corrective Action

Where the intended outcome is not achieved:

Implementation → Verification Failure → Change Reassessment → Additional

Implementation / Recovery / Rejection / Alternative Resolution

Governance Requirements

A conforming SCMS implementation must satisfy the following requirements. Change Identity and Formation

• Establish identifiable software change cases

• Preserve change origin and purpose

• Identify affected software objects

• Establish current and intended state references

• Define change scope and boundaries

• Preserve supporting evidence

• Identify change classification and significance

• Preserve material uncertainty

Impact and Feasibility

• Evaluate direct change impact

- Evaluate material indirect impact
- Identify affected dependencies and configurations
- Identify quality and security implications
- Evaluate operational consequences
- Evaluate change feasibility where applicable
- Identify material alternatives where justified
- Preserve impact evidence and limitations

Authority and Coordination

- Identify applicable change decision authority
- Preserve decision actor eligibility
- Identify affected coordination functions
- Identify related and concurrent changes
- Preserve implementation sequencing requirements
- Identify conflicts and restrictions
- Govern emergency change conditions
- Preserve automation and autonomous agent boundaries

Lifecycle Re-entry and Implementation

- Determine applicable lifecycle re-entry requirements
- Identify affected SLGA™ Parent Standards
- Preserve the basis for evidence reuse
- Identify evidence requiring renewal
- Preserve implementation references
- Identify material implementation deviations
- Require reassessment where applicable
- Preserve release and deployment governance boundaries

Verification and Closure

- Establish change verification requirements
- Compare intended and actual change outcomes
- Preserve verification evidence
- Identify change failure
- Preserve residual obligations
- Establish change outcome classification
- Preserve closure decisions
- Maintain historical change continuity

Governance Outputs

The principal output is:

Software Change Governance Profile

Supporting outputs include:

Software Change Request Profile

Software Change Impact Profile

Software Change Coordination Profile

Software Lifecycle Re-entry Profile

Software Change Closure Profile

Additional governance records may include:

Change Identity Record

Change Classification

Affected Software Object Register

Current–Intended State Comparison

Change Feasibility Assessment

Change Decision Reference

Change Authority Reference

Lifecycle Re-entry Register

Implementation Traceability Record

Change Verification Record

Residual Obligation Register

Change Version History

Change Closure Record

Governance Evidence

SCMS must preserve evidence sufficient to reconstruct material change decisions and outcomes.

Relevant evidence includes:

Original Change Request

Change Origin Evidence

Affected Object References

Impact Assessment

Authority and Decision Records

Coordination Records

Lifecycle Re-entry Decisions

Implementation Evidence

Quality and Security Verification

Release and Deployment References

Observed Outcome

Residual Obligations

Closure Decision

Evidence must remain associated with the change case and the conclusions it supports.

Governance Continuity

Software change governance must preserve continuity when:

Change Ownership Transfers

Implementation Actors Change

Automation Replaces Manual Execution

Change Scope Evolves

Lifecycle Re-entry Expands

Related Changes Are Merged or Separated

Implementation Is Deferred

Verification Is Repeated

Residual Obligations Continue

Continuity requires the change's identity, decision history, evidence, and

unresolved obligations to remain reconstructable.

Cross-Architecture and Lifecycle Boundary

SCMS coordinates change-related governance across SLGA™ without replacing

specialized Parent Standards.

SRS — Software Requirements Standard

Governs changed requirements, scope, and traceability.

SCS — Software Creation Standard

Governs creation and modification of software artifacts.

SDS — Software Dependency Standard

Governs dependency identity, suitability, introduction, and continuity.

SCFS — Software Configuration Standard

Governs configuration baselines, builds, artifact identities, and

configuration changes.

SQS — Software Quality Standard

Governs quality criteria, testing, defects, and quality assessment.

SSS — Software Security Standard

Governs security requirements, testing, vulnerabilities, and remediation.

SRLS — Software Release Standard

Governs release readiness, authorization, records, and withdrawal.

SDPS — Software Deployment Standard

Governs deployment planning, authorization, production version identity,

monitoring, and recovery.

SRTS — Software Retirement Standard

Governs retirement, migration, support termination, and residual

dependencies.

SLCS — Software Lifecycle Control Standard

Governs lifecycle state consistency, transitions, exceptions, control

transfer, and continuity.

SCMS also consumes applicable authority, accountability, autonomous

systems, and operational reality governance references from the wider OOF® methodology where relevant.

Practical Applications

Enterprise Software Development

Coordinating changes across development teams, software products, release

schedules, and production environments.

Continuous Integration and Delivery

Maintaining traceability between change requests, code modifications,

verification, releases, and deployments.

AI-Assisted Software Engineering

Ensuring AI-generated modifications remain associated with legitimate

change requests, assessed impacts, permitted actions, and verified outcomes.

Autonomous Software Agents

Preventing autonomous agents from treating an assigned objective as

unrestricted authority to modify software, dependencies, infrastructure, or production environments.

Cybersecurity Remediation

Coordinating urgent vulnerability corrections while preserving security

verification, release governance, and deployment authorization.

Critical Infrastructure

Ensuring software modifications affecting consequential operations receive

appropriate impact assessment, coordination, verification, and lifecycle control.

Robotics and Embedded Systems

Governing firmware, control logic, configuration, and behavioral changes

with potentially significant physical consequences.

Use Case 1 — A Small Dependency Update Creates a Major Operational

Change

A development team proposes updating a software library to address a minor

compatibility issue.

The modification requires only a few lines of code.

However, the new library version changes authentication behavior and

introduces an incompatible runtime dependency.

SCMS evaluates:

Change Request → Dependency Impact → Security Impact → Configuration

Impact → Quality Requirements → Lifecycle Re-entry

The assessment identifies:

• The technical modification is small

• The dependency impact is material

• Security behavior requires reassessment

• Configuration and quality verification are necessary

• The change requires additional lifecycle governance before release

Small Code Change ≠ Small Governance Impact.

Use Case 2 — An Autonomous Agent Fixes a Defect but Introduces an

Unauthorized Production Change

An autonomous software engineering agent receives an objective to correct a

reported application defect.

The agent modifies the source code and successfully passes local tests.

It then deploys the modified application directly into production using an

available tool.

SCMS evaluates:

Change Objective → Agent Authority → Modified Artifacts → Impact

Assessment → Lifecycle Re-entry → Release Governance → Deployment Authorization

The assessment identifies:

• The defect correction may be technically valid

• The agent's assigned objective does not establish unrestricted

• Required lifecycle verification may be incomplete

• Release and deployment authorization cannot be inferred from

• The production modification requires appropriate governance assessment

execution authority. successful local tests.

Defect Corrected ≠ Change Legitimately Introduced into Production.

Use Case 3 — Multiple Individually Acceptable Changes Create a Combined

Failure

Two teams prepare separate software changes.

One modifies a shared service interface.

The other updates a dependent application.

Both changes pass their individual tests.

When deployed together, their combined behavior produces an operational failure.

SCMS evaluates:

Change A → Change B → Shared Dependencies → Combined Impact → Coordination

→ Integration Verification

The assessment identifies:

• Each change has an identifiable purpose

• Individual verification does not establish combined compatibility

• The changes affect a shared operational dependency

• Coordination and integration verification are required

• The resulting failure creates reassessment and corrective-action

obligations.

Individually Verified Changes ≠ Combined Operational Compatibility.

Critical Distinctions

Change Implemented ≠ Change Governed.

Change Approved ≠ Every Lifecycle Requirement Satisfied.

Technical Modification ≠ Verified Software Improvement.

Change Request ≠ Change Authorization.

Change Justification ≠ Implementation Permission.

Limited Change Scope ≠ Limited Change Impact.

Change Size ≠ Required Governance Depth.

Urgent Change ≠ Automatically Authorized Change.

Previously Verified ≠ Still Verified After Material Change.

Agent Recommendation ≠ Authorized Software Change.

Task Authority ≠ Constituent Action Authority.

Local Test Success ≠ Production Deployment Authorization.

Implementation Complete ≠ Intended Outcome Verified.

Administrative Closure ≠ Successful Change Implementation.

Individually Verified Changes ≠ Combined Operational Compatibility.

Software Change Management ≠ Replacement of Specialized Lifecycle

Governance.

Canonical Boundary

SCMS governs the identification, assessment, coordination, lifecycle

routing, implementation traceability, verification, and closure of software changes across the SLGA™ software lifecycle.

It establishes the relationship between change origin, purpose, affected

software objects, impact, applicable authority, required lifecycle re-entry, implementation evidence, resulting state, and closure.

SCMS does not independently replace software requirements governance, software creation, dependency governance, configuration control, quality verification, security assessment, release authorization, deployment governance, or software retirement.

Its five Core Modules are:

Change Request Management Module (CRMM)

Change Impact Assessment Module (CIAM)

Change Coordination Control Module (CHCM)

Lifecycle Re-entry Control Module (LRECM)

Change Closure Management Module (CCLM)

Its principal governance output is the:

Software Change Governance Profile

Canonical Closing Statement

Software change is not governed merely because a modification was requested, approved, implemented, or successfully deployed.

A minor code edit may introduce significant dependency consequences.

A security correction may create new compatibility problems.

Two independently verified modifications may become incompatible when combined.

An automated pipeline may implement a change without establishing that every affected lifecycle requirement has been satisfied.

An autonomous agent may solve the assigned technical problem while exceeding its authority or introducing unverified operational consequences.

And a completed change may leave unresolved obligations that remain relevant long after implementation.

SCMS establishes the governance structure required to identify and control

these conditions.

It requires software changes to remain associated with their origin,

purpose, affected objects, assessed consequences, legitimate decisions, implementation pathways, and verified outcomes.

It directs changes into the lifecycle mechanisms that are materially affected without unnecessarily repeating unrelated governance activities.

It preserves the distinction between change approval and implementation

authorization, between technical modification and operational improvement, and between implementation completion and evidence-supported closure.

It ensures that emergency changes, automated modifications, and autonomous agent actions remain subject to applicable authority and lifecycle boundaries.

And it maintains continuity when changes evolve, overlap, fail, require reassessment, or produce additional obligations.

A software change must not be judged only by whether the intended modification was made. It must be judged by whether the change was justified, its consequences were adequately assessed, the applicable lifecycle requirements were satisfied, and the resulting software state was legitimately and verifiably established.

Software Change Management Standard (SCMS) transforms software change from

an isolated modification activity into a traceable, coordinated, and evidence-supported governance process that preserves software lifecycle integrity as systems evolve.

Module Architecture

→ Change Request Management Module (CRMM)

→ Change Impact Assessment Module (CIAM)

→ Change Coordination Control Module (CHCM)

→ Lifecycle Re-entry Control Module (LRECM)

→ Change Closure Management Module (CCLM)

Related Documents

→ About Software Change Management Standard (SCMS)